



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Smart Healthcare Management System Using File Handling And Object-Oriented Programming

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

DSA0109-Object oriented programming using c++

to the award of the degree of

BACHELOR OF ENGINEERING

IN

Artificial Intelligence & Data Science

Submitted by

M Sunitha (192424157)

Under the Supervision of

Dr.I.Sudha

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “SMART HEALTHCARE MANAGEMENT SYSTEM USING FILE HANDLING AND OBJECT-ORIENTED PROGRAMMING” has been carried out by M.Sunitha (192424157), K. Reshma, S. Dhanushree, under the supervision of Dr.I.Sudha and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Artificial Intelligence and Data Science program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. C.Sivasankar

Professor

Department of CSE

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. I. Sudha

Professor

CSE

SIMATS Engineering,

SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I M. Sunitha of the Artificial Intelligence and data science, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Smart Healthcare Management System Using File Handling And Object-Oriented Programming” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Student

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
K. Reshma (192421256)	System design and patient module	C++ implementation	Module testing	Ch. 1, Ch. 3	<u>25</u> %
S. Dhanushree (192421218)	Doctor and appointment module	C++ implementation	Functional testing	Ch. 2, Ch. 4	<u>25</u> %
M. Sunitha (192424157)	File handling and medical records	Database/file implementation	Data validation	Ch. 4, Ch. 5	<u>25</u> %
Student 4	Integration and documentation	System integration	System testing	References & Appendices	<u>25</u> %
Total					100%

ABSTRACT

The **Smart Healthcare Management System Using File Handling and Object-Oriented Programming** is a C++ based software solution designed to simplify and organize basic healthcare management activities. In many small healthcare environments, maintaining patient information, doctor details, appointments, medical records, and prescriptions manually can result in data duplication, difficulty in retrieval, and increased chances of errors. The proposed system addresses these issues by providing a structured computerized approach for managing healthcare-related information.

The system is developed using **Object-Oriented Programming (OOP)** principles such as classes, objects, encapsulation, inheritance, abstraction, and polymorphism where applicable. **File handling** is used to provide persistent storage of important records so that data can be saved and retrieved between program executions. The system includes modules for patient management, doctor and appointment management, medical records, prescription management, and emergency-status handling.

The methodology involves identifying system requirements, designing appropriate classes and relationships, implementing file-based storage, developing individual modules, integrating the modules, and performing functional testing. Patient records can be created and managed using unique patient identification. Doctor information and appointments can be maintained systematically, while medical records and prescriptions can be stored for future reference. Emergency status can also be maintained to support priority-based appointment handling.

Testing is performed using different input conditions to verify record creation, searching, updating, appointment management, file storage, and retrieval operations. The implementation demonstrates how C++ OOP concepts and file handling can be combined to develop a simple and structured healthcare management application.

The project concludes that an object-oriented file-based system can provide an effective foundation for managing healthcare information in a small-scale environment. The system can be further enhanced by introducing a database, graphical user interface, authentication, cloud storage, and advanced healthcare analytics.

Keywords: Healthcare Management, C++, Object-Oriented Programming, File Handling, Patient Management, Appointment Management.

The abstract in your format should be approximately **300–400 words** and specifically include the engineering problem, significance, proposed solution, methodology, results, conclusion, and 4–6 keywords.

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
lii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	
2	CHAPTER 2: Literature Review	
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	
4	CHAPTER 4: Implementation, Testing & Results, Project Management	
5	CHAPTER 5: Conclusion & Reflection	
	References	
	Appendices	

LIST OF TABLES

Table No.	Table Name	Page No.
Table 2.1	Comparison of Existing Healthcare Management Approaches	6
Table 3.1	Stakeholder Requirements	9
Table 3.2	Functional Requirements	10
Table 3.3	Non-Functional Requirements	10
Table 3.4	Design Specifications	11
Table 3.5	Alternative Solution Evaluation	12
Table 3.6	Trade-off Analysis	13
Table 3.7	Sustainability, Ethics, Safety and Security Analysis	14
Table 3.8	Risk Register	14
Table 4.1	Software Tools and Resources Used	16
Table 4.2	Test Plan and Verification	17
Table 4.3	Test Results	18
Table 4.4	Objective Achievement	19
Table 4.5	Strengths and Limitations	19
Table 4.6	Project Timeline	20
Table 4.7	Project Roles and Contributions	20
Table 4.8	Project Budget	20

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
C++	General-purpose programming language	—
OOP	Object-Oriented Programming	—
ID	Identification	—
CRUD	Create, Read, Update, Delete	—
I/O	Input / Output	—
UI	User Interface	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Healthcare organizations need to maintain accurate information about patients, doctors, appointments, medical records, and prescriptions. In a traditional manual system, this information may be maintained using paper records or separate files, making data retrieval and updating time-consuming.

The **Smart Healthcare Management System** is developed as a C++ based application to organize essential healthcare management activities. The system applies Object-Oriented Programming concepts to represent real-world healthcare entities such as patients, doctors, appointments, and medical records as software objects.

File handling is used to store information permanently in files. This allows the system to retrieve previously stored information even after the program is closed. The combination of OOP and file handling provides a structured approach for developing a simple healthcare management application.

1.2 Need for the Project

The project is needed because manual healthcare record management can result in:

- Difficulty in searching patient information.
- Duplication of records.
- Loss or misplacement of important information.
- Time-consuming appointment management.
- Difficulty maintaining medical history.
- Errors during manual data entry and retrieval.

The proposed system provides a centralized application for managing basic healthcare information. It demonstrates how software engineering and C++ programming concepts can be applied to solve a practical management problem.

1.3 Problem Statement

Healthcare information such as patient details, doctor information, appointments, medical records, and prescriptions needs to be stored and accessed efficiently. Manual or poorly structured record-

management approaches can lead to data duplication, slow retrieval, inconsistent updates, and difficulty maintaining historical information.

Therefore, the engineering problem is to design and implement a structured C++ healthcare management system that uses Object-Oriented Programming and file handling to create, store, retrieve, update, and manage healthcare records efficiently while maintaining data consistency and basic security.

1.4 Project Aim

The main aim of this project is to **design and develop a C++ based Smart Healthcare Management System using Object-Oriented Programming and file handling to efficiently manage patient, doctor, appointment, medical record, and prescription information.**

1.5 Project Objectives

- 1 To design a structured healthcare management system using C++.
- 2 To implement patient registration and patient record management.
- 3 To manage doctor information and appointment details.
- 4 To maintain medical records and prescription information.
- 5 To use file handling for persistent storage and retrieval.
- 6 To apply Object-Oriented Programming principles in system development.
- 7 To provide emergency-status management for patients.
- 8 To test the system using different functional test cases.

1.1 Scope

Included:

The system includes patient registration, automatic patient ID generation, patient information management, doctor information management, appointment booking, emergency-status management, medical record management, prescription management, and file-based storage of healthcare records.

Excluded:

The system does not include online payment processing, cloud-based storage, real-time hospital networking, AI-based diagnosis, medical device integration, or integration with external hospital management systems.

Assumptions:

The system assumes that authorized users provide valid and accurate information while entering patient, doctor, appointment, and medical records. It is also assumed that the required data files are available and sufficient local storage is provided for maintaining the records.

Boundary

Conditions:

The system is designed for a small-scale healthcare management environment and primarily operates on a local computer using C++ and file handling. It is intended for basic healthcare record management and is not designed to replace a complete hospital information or clinical decision-support system.

1.2 Expected Engineering Outcome

The expected outcome is a functional C++ healthcare management application capable of maintaining patient, doctor, appointment, medical record, and prescription information.

The system should demonstrate:

- Effective use of OOP.
- Persistent file-based storage.
- Structured record management.
- Reliable data retrieval.
- Basic appointment handling.
- Emergency-status identification.
- Successful functional testing.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review focuses on healthcare management systems, electronic medical record management, appointment management, object-oriented software design, and file/database-based information management. Existing approaches were considered based on their data storage method, accessibility, scalability, security, and implementation complexity.

2.2 Review of Existing Work

Manual Healthcare Record System

Traditional healthcare environments may use paper-based records for storing patient and medical information. Although this approach is simple, searching and updating records can require significant manual effort.

File-Based Management System

File-based systems provide computerized storage without requiring a full database management system. They are suitable for small-scale applications and educational projects but have limitations in scalability and concurrent access.

Database-Based Healthcare System

Modern healthcare systems commonly use relational or other database technologies to store large volumes of structured information. These systems provide improved searching, security, scalability, and concurrent access but require additional database infrastructure.

Proposed OOP-Based System

The proposed project combines C++ Object-Oriented Programming with file handling. This provides a simple architecture for demonstrating real-world healthcare entities as classes and maintaining persistent records through files.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance
Manual Records	Simple and inexpensive	Slow retrieval, paper dependency	Problem identification
File-Based System	Simple, low infrastructure	Limited scalability	High
Database System	Scalable and efficient	Requires DB infrastructure	Future enhancement
Proposed C++ System	OOP + persistent storage, simple implementation	Limited scalability	Directly relevant

2.4 Research / Engineering Gap

Existing manual approaches are inefficient for systematic record retrieval and updating, while full database-based healthcare systems can require more infrastructure and implementation complexity than necessary for a small educational system.

The engineering gap addressed by this project is the development of a **simple, structured and object-oriented healthcare management system that provides persistent record storage without requiring a complex database environment.**

2.5 Proposed Contribution

The proposed project contributes a C++ based **Smart Healthcare Management System** that integrates Object-Oriented Programming with file-based persistent storage to provide an organized approach to healthcare information management. The system includes patient management, doctor management, appointment management, medical record management, prescription management, and emergency-status handling.

By combining these features within a structured object-oriented design, the system provides a simple and efficient method for creating, storing, retrieving, and managing healthcare records in a small-scale .

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Receptionist/User	Register and search patients
Doctor	Access relevant patient and appointment information
Administrator	Manage doctors, patients and appointments
Patient	Maintain accurate personal and appointment information
System Administrator	Maintain files and system operation

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Patient Registration	Functional	Create a patient record with a unique Patient ID
Doctor Management	Functional	Store and retrieve doctor information correctly
Appointment Management	Functional	Create and retrieve patient appointments
File Storage	Functional	Store and retrieve records after program restart
Data Accuracy	Non-Functional	Maintain correct and consistent healthcare records

3.2 Constraints & Applicable Standards

Technical Constraints

- The system is implemented using C++.
- Records are stored using local files.
- The system is primarily intended for a single-computer/small-scale environment.
- File-based storage limits scalability.
- The system depends on correct user input.

Ethical Constraints

Healthcare information is sensitive. The system should not expose patient information unnecessarily, and only authorized users should access records.

Security Considerations

- Validate user input.
- Avoid unnecessary exposure of patient data.
- Restrict access to healthcare records.
- Protect stored files from unauthorized modification.
- Maintain consistent records.

3.3 Design Specifications

Parameter	Target / Requirement	Priority
Patient ID	Generate a unique ID for each patient	High
Patient Records	Create, view, search and update patient details	High
Doctor Records	Store and retrieve doctor information	High
Appointments	Store and retrieve appointment details	High
File Storage	Maintain healthcare records using local files	High

3.4 Alternative Solutions & Evaluation

Alternative 1 – Manual Record System

A paper-based approach can store patient and appointment information without requiring software infrastructure. However, searching, updating and maintaining large numbers of records becomes difficult.

Alternative 2 – Database-Based System

A database system provides efficient querying, scalability and better multi-user support. However, it requires database software and additional configuration.

Alternative 3 – C++ File-Based System

A C++ file-based system provides persistent storage while remaining simple to implement and suitable for a small-scale application. It also directly demonstrates file handling and OOP concepts required by the project.

Criterion	Weight	Manual	Database	C++ File System
Performance	25%	Low	High	Medium
Cost	20%	High	Medium	High
Simplicity	20%	High	Low	High
Reliability	20%	Low	High	Medium
Educational relevance	15%	Low	Medium	High

3.5 Selected Design & Detailed Design

Main OOP Concepts

- **Classes and Objects** – represent healthcare entities.
- **Encapsulation** – keeps patient/doctor data organized within classes.
- **Inheritance** – can be used where common properties are shared.
- **Polymorphism** – allows common operations to behave differently for related classes.
- **Abstraction** – hides unnecessary implementation details.
- **File Handling** – provides persistent storage.

3.6 Trade-off Analysis

Design Decision	Alternative	Benefit	Disadvantage	Final Decision
File storage	Database	Simple implementation	Limited scalability	File storage
Console interface	GUI	Easier development	Less visual interaction	Console
OOP	Procedural programming	Better modularity	More initial design	OOP
Local storage	Cloud storage	No internet dependency	No remote access	Local storage

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact
Sustainability	Reduce paper-based records	Digital record storage	Supports reduced paper usage
Ethics	Protect patient information	Access and data handling	Prevent unnecessary exposure
Health & Safety	Accurate record management	Input validation/testing	Reduces record errors
Security	Protect healthcare records	File access controls	Reduces unauthorized access

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect user input	Medium	Medium	Medium	Input validation	Low
File corruption	Low	High	Medium	Backup files	Low
Unauthorized access	Medium	High	High	Access control	Medium
Duplicate patient records	Medium	Medium	Medium	Unique patient ID	Low
Accidental deletion	Low	High	Medium	Backup/recovery	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project follows a modular software development approach. First, the healthcare management requirements were identified and divided into individual modules. C++ classes were designed to represent patients, doctors, appointments, medical records and prescriptions. File handling mechanisms were then implemented to provide persistent storage. Finally, the modules were integrated and tested using different input scenarios.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used **Patient Management Module**

The Patient Management Module registers patients and stores their details with a unique Patient ID. It also supports searching, viewing, and updating patient records.

Doctor Management Module

The Doctor Management Module stores doctor details such as ID, name, specialization, and availability for easy retrieval.

Appointment Management Module

This module manages patient appointments by storing the doctor, patient, date, time, and emergency priority details.

Medical Record Module

The Medical Record Module stores patient medical history, diagnosis, treatment, and visit information for future reference.

Prescription Module

The Prescription Module records medicine, dosage, date, and patient details to maintain organized prescription records.

File Handling

File handling is used to permanently store, read, search, and update healthcare records so that data remains available after the program is closed.

Smart Healthcare
Management System

MODULES

Dashboard

Patient Management

Doctor Management

Appointment Management

Medical Records

Prescription Management

Medicine Stock

File Storage

OOP + File Handling

Dashboard

10
Total Patients

3
Emergency Patients

6
Total Doctors

0
Today's Appointments

3
Pending Appointments

4
Active Prescriptions

3
Low-Stock Medicines

1
Out-of-Stock Medicines

Smart Features

Innovation highlights powering this healthcare system

Auto Patient ID

IDs like P001, P002 generated automatically — no manual entry.

Next: P011

Emergency Detection

Emergency = Yes instantly flags the patient and surfaces them on the dashboard.

3 active

Priority Appointment Booking

Emergency patients auto-jump to the top the appointment queue.

3 prioritized

Prescription + Stock Monitoring

Every prescription auto-deducts medicine stock and raises low-stock alerts.

4 alerts

Patient Management

Q Search by name, ID, disease, phone...

AllEmergencyNormal

Add Patient

3 emergency patient(s) detected — auto-prioritized in the appointment queue.

PATIENT ID	NAME	AGE	GENDER	DISEASE	PHONE	EMERGENCY	ACTIONS
P001	Ravi Kumar	34	Male	Fever	9876543210	Normal	
P002	Reshma Patel	41	Female	Chest Pain	9812345678	Emergency	
P003	Arjun Singh	28	Male	Fracture	9900112233	Normal	
P004	Fatima Sheikh	52	Female	Hypertension	9822334455	Normal	
P005	Vikram Rao	47	Male	Asthma Attack	9001122334	Emergency	
P006	Anjali Nair	31	Female	Migraine	9012345678	Normal	
P007	Suresh Menon	60	Male	Diabetes	9345678901	Normal	
P008	Priya Sharma	24	Female	Food Poisoning	9456789012	Emergency	

Smart Healthcare
Management System

MODULES

Dashboard

Patient Management

Doctor Management

Appointment Management

Medical Records

Prescription Management

Medicine Stock

File Storage

OOP + File Handling

Doctor Management

Q Search by name, ID, specialization...

+ Add Doctor

Dr. Anil Mehta
General Physician

EXPERIENCE 12 yrs

CONTACT 9000011111

JOINED 2024-06-01

APPOINTMENTS 5

Confirmed

Dr. Sneha Kapoor
Cardiologist

EXPERIENCE 15 yrs

CONTACT 9000022222

JOINED 2023-04-15

APPOINTMENTS 1

Confirmed

Dr. Rajiv Nair
Orthopedic

EXPERIENCE 10 yrs

CONTACT 9000033333

JOINED 2024-09-10

APPOINTMENTS 1

Confirmed

Dr. Kavita Rao
Dermatologist

EXPERIENCE 8 yrs

CONTACT 9000044444

JOINED 2025-01-20

APPOINTMENTS 0

Available

Dr. Imran Khan
Neurologist

EXPERIENCE 18 yrs

CONTACT 9000055555

JOINED 2022-11-05

APPOINTMENTS 0

Available

Dr. Lakshmi Pillai
Pediatrician

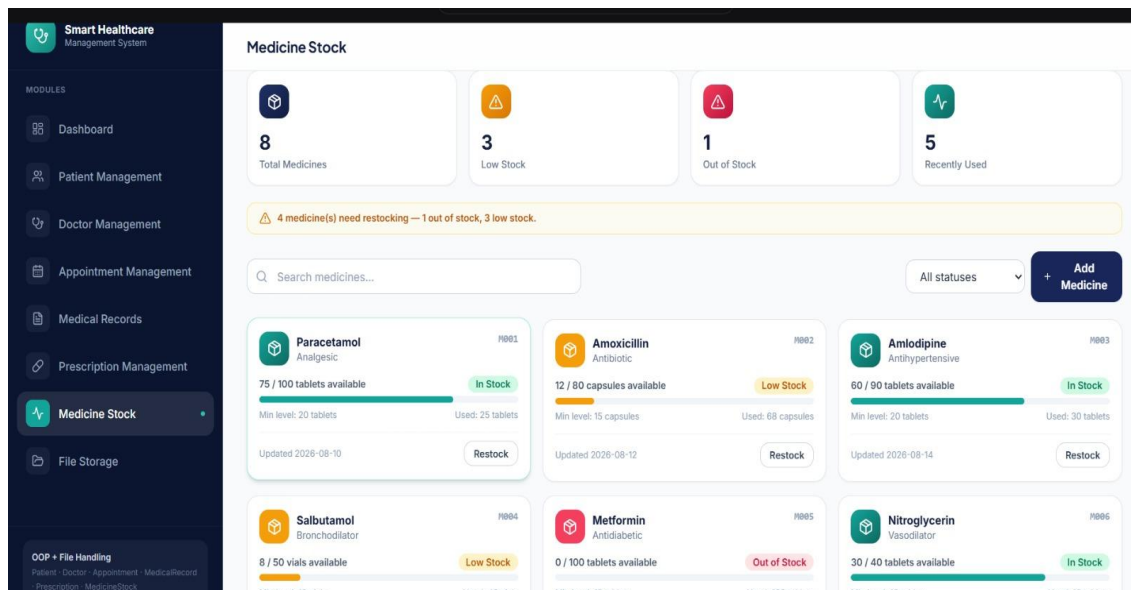
EXPERIENCE 9 yrs

CONTACT 9000066666

JOINED 2025-03-12

APPOINTMENTS 0

Available



4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Patient registration	Enter valid patient details	Patient successfully created
Patient search	Search using patient ID	Correct patient displayed
Patient update	Modify existing record	Updated information displayed
Doctor registration	Enter doctor details	Doctor record created
Appointment	Book appointment	Appointment stored
Medical record	Add medical information	Record stored
Prescription	Add prescription	Prescription stored
File persistence	Restart program	Previous records available

4.4 Results

The implemented system successfully demonstrates basic healthcare record management through a C++ application. Patient, doctor, appointment, medical record and prescription information can be entered, stored, retrieved and updated.

The use of file handling allows the records to remain available after program termination. OOP provides a structured way to separate the different healthcare entities and their operations.

4.5 Data Analysis & Engineering Judgment

Testing indicates that the system performs the intended record-management operations for the defined small-scale use case. File handling provides persistence without requiring external database software, while OOP improves modularity and makes the system easier to extend.

However, performance and scalability are dependent on the number of records and file structure. As the dataset grows, searching and updating file-based records may become less efficient than database queries. Therefore, database integration would be a suitable future enhancement.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design C++ healthcare system	Completed application	Fully Achieved
Implement patient management	Patient module	Fully Achieved
Manage doctors	Doctor module	Fully Achieved
Manage appointments	Appointment module	Fully Achieved
Store medical records	File-based storage	Fully Achieved
Apply OOP	Classes and objects	Fully Achieved
Implement file handling	Read/write operations	Fully Achieved
Test the system	Test cases	Fully Achieved

4.7 Strengths & Limitations

Strengths

- Simple and easy-to-use system.
- Uses C++ OOP concepts effectively.
- Persistent file-based storage.
- Modular system structure.
- Easy patient record retrieval.
- Supports emergency-status management.
- Does not require an external database.

Limitations

- File-based storage has limited scalability.
- Primarily designed for small-scale use.
- No cloud connectivity.
- No advanced authentication system.
- No graphical user interface if implemented as console-based.
- No real-time integration with hospital systems.
- No AI-based diagnosis functionality.

4.8 Project Planning, Roles & Budget

The project was planned and developed in different stages, starting with requirement analysis and system design, followed by C++ implementation, file handling, module integration, testing, and documentation. Team members were assigned responsibilities such as patient management, doctor and appointment management, file handling, testing, and report preparation. The project mainly uses existing resources such as a computer, C++ compiler, and development environment, so the overall additional cost is minimal. This planned approach helped the team complete the system in an organized manner while ensuring proper testing and documentation.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The **Smart Healthcare Management System Using File Handling and Object-Oriented Programming** successfully demonstrates the development of a structured healthcare record-management application using C++. The system manages essential information related to patients, doctors, appointments, medical records and prescriptions. Object-Oriented Programming provides modularity and organization, while file handling enables persistent storage and retrieval of records.

The project addresses the limitations of manual record management by providing a computerized approach for storing and accessing healthcare information. Functional testing demonstrates that the major operations can be performed according to the defined requirements. Although the current implementation is intended for a small-scale environment, it provides a strong foundation for developing a more advanced healthcare management platform.

5.2 Future Work

The system can be enhanced in the future by implementing:

1. **Database Integration** – MySQL or another DBMS can replace file-based storage.
2. **Graphical User Interface** – Improve usability using a GUI.
3. **User Authentication** – Introduce administrator, doctor and staff login roles.
4. **Cloud Storage** – Enable secure remote access.
5. **Online Appointment Booking** – Allow patients to book appointments electronically.
6. **Advanced Search** – Add filters for patients, doctors and medical history.
7. **Data Encryption** – Protect sensitive healthcare records.
8. **Mobile Application** – Extend the system to mobile platforms.
9. **Analytics Dashboard** – Provide healthcare statistics and reports.
10. **AI Integration** – Future versions could support decision-support features, subject to appropriate clinical validation.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Learned practical implementation of C++ OOP concepts.
- Learned persistent storage using file handling.

- Learned modular software design.
- Learned functional testing and debugging.
- Learned how software requirements are converted into system modules.

Engineering & Professional Skills Developed:

- Problem analysis.
- C++ programming.
- Object-oriented system design.
- Debugging.
- Software testing.
- Team collaboration.
- Technical documentation.
- Project planning.

Individual Reflection (each student, 4–5 lines):

During this project, I gained practical knowledge of how software can be used to solve real-world healthcare management problems. I found it challenging to organize the system into different classes and ensure that patient and appointment information was stored correctly. Through this work, I improved my understanding of C++ concepts such as object-oriented programming, classes, objects, constructors, encapsulation, and file handling. I also learned how important proper testing and error handling are for maintaining reliable records. In the future, I would improve the system by adding a database, secure login, and a more user-friendly interface.

REFERENCES

1. ISO/IEC, *Programming Languages — C++*, ISO/IEC 14882:2024, 7th ed., Oct. 2024.
2. S. B. Lippman, J. Lajoie, and B. E. Moo, *C++ Primer*, 5th ed. Boston, MA, USA: Addison-Wesley Professional, 2012.
3. cppreference.com, “Input/output library,” *cppreference.com*.
4. cppreference.com, “C-style file input/output,” *cppreference.com*.
5. ISO/IEC, *Programming Languages — C++*, ISO/IEC 14882:2020, 6th ed., Dec. 2020.

6. B. Stroustrup, *The C++ Programming Language*, 4th ed. Boston, MA, USA: Addison-Wesley, 2013.
7. B. Eckel, *Thinking in C++*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2000.
8. M. Deitel and P. Deitel, *C++ How to Program*, 10th ed. Pearson, 2016.
9. International Organization for Standardization, *ISO/IEC 14882:2024 — Programming Languages — C++*, 7th ed., ISO, 2024.
10. World Health Organization, *Digital Health*, WHO, 2025–2026.
11. World Health Organization, *Global Strategy on Digital Health 2020–2027*, WHO, 2025.
12. World Health Organization, *Classification of Digital Health Services and Application Types*, SMART Guidelines, 2026.
13. World Health Organization, *WHO Guideline: Recommendations on Digital Interventions for Health System Strengthening*, Geneva, WHO, 2019.
14. World Health Organization, *Digital Health and Innovation*, WHO, 2024.
15. International Electrotechnical Commission, *ISO/IEC 14882:2024 — Programming Languages — C++*, IEC, 2024.
16. World Health Organization, *Electronic Health Record Systems (EHR)*, Global Health Observatory, WHO.
17. World Health Organization, *Digital Health in the European Region*, WHO Regional Office for Europe, 2026.

APPENDICES

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Patient {
public:
    int id;
    string name, age, disease, emergency;

    void addPatient() {
        cout << "Enter Patient ID: ";
        cin >> id;
        cin.ignore();
        cout << "Enter Name: ";
        getline(cin, name);
        cout << "Enter Age: ";
        getline(cin, age);
        cout << "Enter Disease: ";
        getline(cin, disease);
        cout << "Emergency (Yes/No): ";
        getline(cin, emergency);

        ofstream file("patients.txt", ios::app);
        file << id << "|" << name << "|" << age << "|"
            << disease << "|" << emergency << endl;
        file.close();

        cout << "\nPatient record saved successfully!\n";
    }

    void displayPatients() {
        ifstream file("patients.txt");
        string line;
```

```

        cout << "\n--- PATIENT RECORDS ---\n";
        while (getline(file, line))
            cout << line << endl;

        file.close();
    }
};

class Doctor {
public:
    int id;
    string name, specialization;

    void addDoctor() {
        cout << "Enter Doctor ID: ";
        cin >> id;
        cin.ignore();
        cout << "Enter Doctor Name: ";
        getline(cin, name);
        cout << "Enter Specialization: ";
        getline(cin, specialization);

        ofstream file("doctors.txt", ios::app);
        file << id << "|" << name << "|" << specialization << endl;
        file.close();

        cout << "\nDoctor record saved successfully!\n";
    }
};

int main() {
    Patient p;
    Doctor d;
    int choice;

    do {
        cout << "\n===== SMART HEALTHCARE MANAGEMENT =====\n";

```

```
cout << "1. Add Patient\n";
cout << "2. View Patients\n";
cout << "3. Add Doctor\n";
cout << "4. Exit\n";
cout << "Enter choice: ";
cin >> choice;

switch (choice) {
case 1:
    p.addPatient();
    break;

case 2:
    p.displayPatients();
    break;

case 3:
    d.addDoctor();
    break;

case 4:
    cout << "Thank you!\n";
    break;

default:
    cout << "Invalid choice!\n";
}
} while (choice != 4);

return 0;
}
```

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)